2026

Presented by
TEMEN
ALEMAYEHU

Presented to
FUTURE INTERNS

[API Security Risk Analysis (Modern SaaS Skill)]

2026

API Security Testing Report

1. Introduction

Let me start by giving you some background information about APIs.

- APIs are used by software systems to talk to each other.
- **For example** when you check the weather on your phone or use a banking app to view your balance APIs are at work.
- If APIs are not built with security in mind they can be very dangerous.
- I mean think about it APIs can be used to get to your information.
- In this report I will walk you through a security analysis I performed on an API.
- I used Postman on Kali Linux to perform the analysis.
- My goal was to think like a hacker and find any security gaps in the API.
- I looked for API vulnerabilities such as:
    - ✓ No login required to use the API
    - ✓ Being able to see users data without permission
    - ✓ Getting too much sensitive information from the API
    - ✓ Being able to send bad inputs to the API without it checking
- All testing was done safely using a public API designed for learning and testing.\
- No real user data was used.

2. API Information

- ➢ **API Name:** JSONPlaceholder
- ➢ **Base URL:** https://jsonplaceholder.typicode.com
- ➢ **Tool Used:** Postman
- ➢ **Operating System:** Kali Linux

A note about JSONPlaceholder:

- ✓ It is an API that does not store real user data.
- ✓ It is meant for testing and learning about API security.
- ✓ It is like a simulation.
- ✓ It behaves like an API. Does not put anyone's personal information at risk.

3. Testing Methodology

Here is how I tested the API.

I performed the following steps:

1. I sent requests to retrieve public user data.
2. I changed the URL parameters manually to see if the API would stop me.
3. I sent inputs to see how the API would react.
4. I sent requests with user data to see if the API would accept it.

After each request I carefully looked at the server's response to spot any vulnerability.

4. Security Testing and Findings

Now let's get into what I discovered.

4.1 No Authentication

- ✓ **What I did:** I accessed the API without providing any login information.

- ✓ **What happened:** The API returned all user data.
- ✓ **Why this matters:** Imagine an API for a banking app that did not ask you to log in. Anyone on the internet could see your account details. The API does not verify who is making the request. No authentication means no security.
- ✓ **Risk level:** Medium to High

4.2 Broken Authorization

- ✓ **What I did:** I changed the user ID in the URL manually.
- ✓ **What happened:** Each time I got back a user's information.
- ✓ **Why this matters:** The API assumes that just because you can access a user's data you should be allowed to. In a system the API should check if the person asking for the data is allowed to see it.
- ✓ **Risk level:** Medium to High

4.3 Excessive Data Exposure

- ✓ **What I did:** I looked at the data returned for an user.
- ✓ **What happened:** The API sent back not the username, but also the email address, phone number and full address.
- ✓ **Why this matters:** Do you really need to expose someone phone number and address just to show their name? This is called data exposure. In an app a hacker could harvest information just by calling this API.
- ✓ **Risk level:** Medium to High

4.4 Lack of Input Validation

- ✓ **What I did:** I requested a user that does not exist.
- ✓ **What happened:** of a proper error message the API returned an empty response.
- ✓ **Why this matters:** Weak input validation tells me the API is not carefully checking what it receives. In cases this could lead to crashes, information leaks or even injection attacks.
- ✓ **Risk level:** Medium to High

4.5 Unrestricted Data Creation

- ✓ **What I did:** I sent a request to create an user with some fake data.
- ✓ **What happened:** The API accepted the data. Returned it back to me as if it were saved.
- ✓ **Why this matters:** There was no authentication, no validation, no filtering. Anyone could create data. In a system this could lead to spam, fake accounts or database pollution.
- ✓ **Risk level:** Medium to High

5. Conclusion

This API shows real-world security weaknesses.

- No authentication, authorization, much data exposed, weak input checks and open data creation.
- If a real production API had these issues it would be a disaster waiting to happen.
- A hacker could steal user data impersonate others or flood the system with data.
- The good news is that all of these issues are preventable, with security design.

6. Recommendations

Here is what I would suggest to make this API more secure:

6.1. Add authentication.

Use API keys or tokens.

No token no access.

6.2. Enforce authorization.

Check if the person asking for data is allowed to see it.

6.3. Limit exposed data.

Only return what is necessary.

6.4. Validate all inputs.

Check for values or malicious payloads.

6.5. Return error messages.

Do not leak system details.